

Data Analysis with Python

Lesson 6: Data Visualization with Matplotlib

Data Visualization with Matplotlib - Study Notes

Key Concepts

1. **Figure vs Axes** – A *Figure* is the entire window or page; *Axes* are the individual plots (subplots) inside that figure.
2. **Plotting Functions** – `plot()`, `bar()`, `scatter()`, and `hist()` are the core functions for line, bar, scatter, and histogram plots, respectively.
3. **Styling and Customization** – Titles, labels, legends, color maps, markers, and line styles can be set via keyword arguments or through `matplotlib.rcParams`.
4. **Layout Management** – `subplots()`, `subplot2grid()`, and `GridSpec` allow multiple plots in one figure; `tight_layout()` and `constrained_layout=True` help avoid overlap.
5. **Saving and Exporting** – `savefig()` writes the figure to disk in various formats (PNG, PDF, SVG, etc.) with control over DPI and bounding boxes.

Summary

Matplotlib is the foundational plotting library in Python, providing a flexible, low level API that lets you build almost any type of visual representation. For beginners, mastering the four most common plot types—line, bar, scatter, and histogram—covers a large portion of everyday data exploration tasks. Each plot type is created with a single function call, but the real power comes from customizing the appearance and layout to convey insights clearly.

A line plot (`plot()`) is ideal for time series or continuous data, while a bar chart (`bar()`) excels at comparing discrete categories. Scatter plots (`scatter()`) reveal relationships and correlations between two variables, and histograms (`hist()`) expose the distribution of a single variable. By combining these plots in a single figure and fine tuning labels, colors, and legends, you can produce publication ready visualizations that communicate your findings effectively.

Important Points

- **Importing**

```
```python
import matplotlib.pyplot as plt
```
```

- **Creating a Figure & Axes**

```
```python
fig, ax = plt.subplots() # single plot
fig, axes = plt.subplots(2, 2) # grid of 4 plots
```
```

- **Line Plot**

```

```python
ax.plot(x, y, color='blue', marker='o', linestyle='--',
 label='Sample Line')
ax.set_xlabel('X axis')
ax.set_ylabel('Y axis')
ax.set_title('Line Plot Example')
ax.legend()
```



- Bar Chart



```

```python
ax.bar(categories, values, color='green', width=0.6,
        edgecolor='black')
```


- Scatter Plot


```

```python
ax.scatter(x, y, c=colors, s=sizes, alpha=0.7,
 edgecolors='k', label='Data Points')
```



- Histogram



```

```python
ax.hist(data, bins=20, color='purple',
        edgecolor='black', alpha=0.75)
```


- Styling
 - `title()`, `xlabel()`, `ylabel()`
 - `legend()`, `grid(True)`
 - `tick_params()`, `xticks()`, `yticks()`
 - `rcParams` for global defaults
- Layout


```

```python
plt.tight_layout()
or
fig.tight_layout(pad=3.0)
```



- Saving



```

```python
plt.savefig('plot.png', dpi=300, bbox_inches='tight')
```


- Interactive Backends
 - Jupyter: `%matplotlib inline` or `%matplotlib notebook`
 - Desktop: `plt.show()` to display

```


```


```


```


```


```

Examples

1. Multi Plot Figure (Line, Bar, Scatter, Histogram)

```
```python
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

## Sample data

```
x = np.arange(10)
```

```
y = np.random.rand(10) * 10
```

```
categories = ['A', 'B', 'C', 'D', 'E']
```

```
values = [5, 7, 3, 8, 6]
```

```
points = np.random.randn(50, 2)
```

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8))
```

## Line plot

```
axes[0, 0].plot(x, y, marker='o', color='tab:blue')
```

```
axes[0, 0].set_title('Line Plot')
```

```
axes[0, 0].set_xlabel('X')
```

```
axes[0, 0].set_ylabel('Y')
```

## Bar chart

```
axes[0, 1].bar(categories, values, color='tab:orange')
```

```
axes[0, 1].set_title('Bar Chart')
```

```
axes[0, 1].set_ylabel('Values')
```

## Scatter plot

```
axes[1, 0].scatter(points[:, 0], points[:, 1], c='tab:green',
 alpha=0.6, edgecolor='k')
```

```
axes[1, 0].set_title('Scatter Plot')
```

```
axes[1, 0].set_xlabel('X')
```

```
axes[1, 0].set_ylabel('Y')
```

## Histogram

```
axes[1, 1].hist(points[:, 0], bins=15, color='tab:red',
 edgecolor='black', alpha=0.7)
```

```
axes[1, 1].set_title('Histogram')
```

```
axes[1, 1].set_xlabel('X')
```

```
axes[1, 1].set_ylabel('Frequency')
```

```
fig.tight_layout()
```

```
plt.show()

```

\*Explanation:\*

- ``subplots(2, 2)`` creates a 2x2 grid of Axes.
- Each subplot is customized with titles, labels, and colors.
- ``tight_layout()`` automatically adjusts spacing so labels don't overlap.

## 2. Customizing a Histogram with Density and Multiple Bins

```
```python
import matplotlib.pyplot as plt
import numpy as np
data = np.random.normal(loc=0, scale=1, size=1000)
fig, ax = plt.subplots(figsize=(6, 4))
```

Histogram with density=True to show probability density

```
ax.hist(data, bins=30, density=True, color='steelblue',
        edgecolor='black', alpha=0.8)
```

Overlay a normal distribution curve

```
x = np.linspace(-4, 4, 200)
pdf = (1/np.sqrt(2*np.pi)) * np.exp(-x**2/2)
ax.plot(x, pdf, color='red', lw=2, label='Normal PDF')
ax.set_title('Histogram with Density')
ax.set_xlabel('Value')
ax.set_ylabel('Density')
ax.legend()
plt.tight_layout()
plt.show()
---
```

Explanation:

- ``density=True`` normalizes the histogram so the area sums to 1.
- Overlaying a theoretical PDF helps assess how well the data follow a normal distribution.
- The ``alpha`` parameter makes the bars slightly transparent, allowing the curve to be visible.

Quick Review

1. ****What is the difference between a Figure and an Axes in Matplotlib?****
2. ****Which function would you use to create a bar chart, and what keyword argument controls the bar width?****
3. ****How can you add a legend to a plot, and why might you want to specify ``label`` in the plotting function?****
4. ****Explain the purpose of ``tight_layout()`` and give an example scenario where it is essential.****

5. **Describe how to save a figure with a resolution of 300 DPI and no extra whitespace around the plot.**

Further Reading

- **Advanced Plotting** – `matplotlib.axes.Axes`` methods (``set_xticks``, ``set_yticks``, ``set_ylim``, etc.)
- **Styling** – `matplotlib.rcParams``, custom style sheets (``style.use('ggplot')``)
- **Interactivity** – ``mplcursors``, ``plotly`` integration, and ``bokeh`` for web based plots
- **Seaborn** – High level interface built on Matplotlib for statistical graphics
- **Animation** – `matplotlib.animation.FuncAnimation`` for dynamic visualizations
- **3 D Plotting** – ``mpl_toolkits.mplot3d`` for 3 D scatter, surface, and wireframe plots

These topics will deepen your understanding of Matplotlib and broaden the range of visualizations you can create. Happy plotting!

